```
#include <stdio.h>

int gcd(int a, int b) {
	while (b != 0) {
		int t = b;
		b = a % b;
		a = t;
	}
	return a;
}

int lcm(int a, int b) {
	return (a / gcd(a, b)) * b;
}

int main() {
	int n;
	printf("Enter number of tasks: ");
	scanf("%d", &n);

	int C[20], P[20];
	int idx[20];

	for (int i = 0; i < n; i++) {
		printf("Enter computation time C%d: ", i + 1);
```

```
        scanf("%d", &C[i]);
        printf("Enter period P%d: ", i + 1);
        scanf("%d", &P[i]);
        idx[i] = i;
    }

    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (P[idx[j]] < P[idx[i]]) {
                int temp = idx[i];
                idx[i] = idx[j];
                idx[j] = temp;
            }
        }
    }

    int hyperperiod = P[0];
    for (int i = 1; i < n; i++) {
        hyperperiod = lcm(hyperperiod, P[i]);
    }

    int rem[20] = {0};

    printf("\nSchedule from time 0 to %d:\n", hyperperiod);
    printf("Time\tTask\n");

    for (int current_time = 0; current_time < hyperperiod; current_time++) {
        for (int k = 0; k < n; k++) {
```

```
            int t = idx[k];
            if (current_time % P[t] == 0) {
                rem[t] = C[t];
            }
        }

        int chosen_task = -1;
        for (int k = 0; k < n; k++) {
            int t = idx[k];
            if (rem[t] > 0) {
                chosen_task = t;
                break;
            }
        }

        if (chosen_task != -1) {
            rem[chosen_task]--;
            printf("%d\tT%d\n", current_time, chosen_task + 1);
        } else {
            printf("%d\tIDLE\n", current_time);
        }
    }

    return 0;
```

```
C:\Users\Admin\Desktop\1wa

Enter number of tasks: 2
Enter computation time C1: 20
Enter period P1: 50
Enter computation time C2: 35
Enter period P2: 100

Schedule from time 0 to 100:
Time    Task
0       T1
1       T1
2       T1
3       T1
4       T1
5       T1
6       T1
7       T1
8       T1
9       T1
10      T1
11      T1
12      T1
13      T1
14      T1
15      T1
16      T1
17      T1
18      T1
19      T1
20      T2
```

}

```
#include <stdio.h>

#define INF 1000000000

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
int arrival[50], burst[50], deadline[50];
int remaining[50];
int completed = 0;
int time = 0;

for (int i = 0; i < n; i++) {
    printf("Enter arrival, burst, deadline for P%d: ", i + 1);
    scanf("%d %d %d", &arrival[i], &burst[i], &deadline[i]);
    remaining[i] = burst[i];
}

printf("\nExecution timeline:\n");

while (completed < n) {
    int min_deadline = INF;
    int selected = -1;

    for (int i = 0; i < n; i++) {
        if (arrival[i] <= time && remaining[i] > 0) {
            if (deadline[i] < min_deadline) {
                min_deadline = deadline[i];
                selected = i;
            }
        }
    }

    if (selected == -1) {
        printf("%d ms : CPU is idle.\n", time);
```

```
            time++;
        } else {
            printf("%d ms : Task %d is running.\n", time, selected + 1);
            remaining[selected]--;
            time++;

            if (remaining[selected] == 0) {
                completed++;
            }
        }
    }

    return 0;
}
```

```
Enter number of processes: 3
Enter arrival, burst, deadline for P1: 20 30 10
Enter arrival, burst, deadline for P2: 30 40 60
Enter arrival, burst, deadline for P3: 50 70 30

Execution timeline:
0 ms : CPU is idle.
1 ms : CPU is idle.
2 ms : CPU is idle.
3 ms : CPU is idle.
4 ms : CPU is idle.
5 ms : CPU is idle.
6 ms : CPU is idle.
7 ms : CPU is idle.
8 ms : CPU is idle.
9 ms : CPU is idle.
10 ms : CPU is idle.
11 ms : CPU is idle.
12 ms : CPU is idle.
13 ms : CPU is idle.
14 ms : CPU is idle.
15 ms : CPU is idle.
16 ms : CPU is idle.
17 ms : CPU is idle.
18 ms : CPU is idle.
19 ms : CPU is idle.
20 ms : Task 1 is running.
21 ms : Task 1 is running.
22 ms : Task 1 is running.
23 ms : Task 1 is running.
24 ms : Task 1 is running.
25 ms : Task 1 is running.
26 ms : Task 1 is running.
27 ms : Task 1 is running.
28 ms : Task 1 is running.
29 ms : Task 1 is running.
30 ms : Task 1 is running.
31 ms : Task 1 is running.
32 ms : Task 1 is running.
33 ms : Task 1 is running.
34 ms : Task 1 is running.
35 ms : Task 1 is running.
36 ms : Task 1 is running.
37 ms : Task 1 is running.
38 ms : Task 1 is running.
39 ms : Task 1 is running.
40 ms : Task 1 is running.
```